Preparation Notebook

Setup Environment

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

Student Information

In [ ]:
```
# <Student to fill this section and then remove this comment>
group_name = "3246106-26AU-AT3-Group "
student_name = "Nana Ama Goldwater"
student_id = "26137455"
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='group_name', value=group_name)
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_name', value=student_name)
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_id', value=student_id)
```

---

0. Python Packages

0.a Install Additional Packages

> If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

In [ ]:
```
# No additional packages required. scikit-learn, pandas, numpy and altair
# are already available in the Colab runtime.
```

0.b Import Packages

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
import pandas as pd
import altair as alt
```

A. Feature Selection

A.0 Load Data

In [ ]:
```python
# DO NOT MODIFY THE CODE IN THIS CELL
# Load datasets using the correct local path
try:
  customer_df = pd.read_csv("/content/customer.csv")
  person_df = pd.read_csv("/content/person.csv")
  product_category_df = pd.read_csv("/content/product_category.csv")
  product_cost_history_df = pd.read_csv("/content/product_cost_history.csv")
  product_list_price_history_df = pd.read_csv("/content/product_list_price_history.csv")
  product_sub_category_df = pd.read_csv("/content/product_sub_category.csv")
  product_df = pd.read_csv("/content/product.csv")
  sales_order_detail_df = pd.read_csv("/content/sales_order_detail.csv")
  sales_order_header_df = pd.read_csv("/content/sales_order_header.csv")
  sales_territory_df = pd.read_csv("/content/sales_territory.csv")
  # special_offer_product_df is omitted as the file is missing in /content/
  special_offer_df = pd.read_csv("/content/special_offer.csv")
  store_df = pd.read_csv("/content/store.csv")
  unit_measure_df = pd.read_csv("/content/unit_measure.csv")
  print("Datasets loaded successfully from /content/ (special_offer_product omitted)")
except Exception as e:
  print(f"Error loading datasets: {e}")
```

A.1 Approach 1 — Use-case-driven table selection

In [ ]:
```python
# Use case: predict whether a customer will place an order in the next 90 days
# (binary classification). The target sits at the CUSTOMER level, and the only
# transactional signal available is order history.

import re
import pandas as pd
import numpy as np
import pathlib
import os

# --- FIX: Explicitly define and load dataframes ---
def load_safe(filename):
    path = f"/content/{filename}"
    if os.path.exists(path):
        return pd.read_csv(path)
    else:
        print(f"Warning: {filename} not found. Returning empty DataFrame.")
        return pd.DataFrame()

customer_df = load_safe("customer.csv")
person_df = load_safe("person.csv")
product_category_df = load_safe("product_category.csv")
product_cost_history_df = load_safe("product_cost_history.csv")
product_list_price_history_df = load_safe("product_list_price_history.csv")
product_sub_category_df = load_safe("product_sub_category.csv")
product_df = load_safe("product.csv")
sales_order_detail_df = load_safe("sales_order_detail.csv")
sales_order_header_df = load_safe("sales_order_header.csv")
sales_territory_df = load_safe("sales_territory.csv")
special_offer_df = load_safe("special_offer.csv")
store_df = load_safe("store.csv")
unit_measure_df = load_safe("unit_measure.csv")

# CamelCase / PascalCase  ->  snake_case.
def to_snake_case(name: str) -> str:
    s = name.strip().replace(' ', '_')
    s = re.sub(r'(.)([A-Z][a-z]+)', r'\1_\2', s)
    s = re.sub(r'([a-z0-9])([A-Z])', r'\1_\2', s)
    return s.lower()

def to_snake(df: pd.DataFrame) -> pd.DataFrame:
    if df.empty: return df
    df = df.copy()
    df.columns = [to_snake_case(c) for c in df.columns]
    return df

cust  = to_snake(customer_df)
sales = to_snake(sales_order_header_df)

print('\ncustomer columns       :', list(cust.columns))
print('sales_order_header cols:', list(sales.columns))
print(f'customer rows           : {len(cust):,}')
```

```
print(f'sales_order_header rows : {len(sales):,}')
```

In [ ]:
```
# Quick assessment of every source table to justify which to keep / drop.
# `customer` and `sales_order_header` are kept; everything else is excluded
# for this use case.
all_tables = [
    ('customer',                  cust,                          True ),
    ('sales_order_header',        sales,                         True ),
    ('person',                    person_df,                     False),
    ('product',                   product_df,                    False),
    ('product_category',          product_category_df,           False),
    ('product_sub_category',      product_sub_category_df,       False),
    ('product_cost_history',      product_cost_history_df,       False),
    ('product_list_price_history', product_list_price_history_df, False),
    ('sales_order_detail',        sales_order_detail_df,         False),
    ('sales_territory',           sales_territory_df,            False),
    ('special_offer',             special_offer_df,              False),
    # special_offer_product_df is excluded as the source file is missing
    ('store',                     store_df,                      False),
    ('unit_measure',              unit_measure_df,               False),
]
summary = pd.DataFrame([
    {'table': name, 'rows': len(df), 'cols': df.shape[1], 'kept_for_use_case': kept}
    for name, df, kept in all_tables
]).sort_values(['kept_for_use_case', 'rows'], ascending=[False, False]).reset_index(drop=True)
display(summary)
```

In [ ]:
```
feature_selection_1_insights = """
Approach 1 :Use case-driven table selection.

The prediction target is defined at the customer level (will this customer
place an order in the next 90 days?). The signal needed is:
  1. Customer attributes (who they are): from `customer`.
  2. Order history (what they've done): from `sales_order_header`.

All other tables in the source schema describe products, promotions,
geography detail, or line-item-level pricing. They could improve a more
sophisticated model (e.g. product-category preference features), but for a
first pass they:
  - Add hundreds of columns of mostly product/SKU detail.
  - Require many-to-one joins that explode row counts and risk leakage if
    not aggregated carefully.
  - Are not directly tied to the customer-level repeat-purchase signal.

`unit_measure` was explored in the EDA notebook and confirmed as a
product-side lookup (38 rows of unit labels) with no direct relationship to
customer behaviour excluded.

Tables retained after Approach 1: `customer`, `sales_order_header`.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_1_insights', value=feature_selection_1_insights)
```

A.2 Approach 2 — Anti-leakage column filter

In [ ]:
```
# Within the two retained tables, eliminate columns that are either
# (a) post-prediction-time and would leak future information into features, or
# (b) free-text / GUID columns that carry no useful signal at the customer
#     level.
candidate_sales_cols = list(sales.columns)
candidate_cust_cols  = list(cust.columns)

# Common AdventureWorks column patterns to drop on leakage / no-signal grounds.
leakage_or_noise_substrings = [
    'rowguid', 'modified_date', 'sales_order_number',
    'purchase_order_number', 'account_number', 'credit_card',
    'currency_rate', 'sales_person', 'comment',
    'ship_date',          # post-order timestamp; would leak future info
    'ship_method',        # determined at fulfilment, not order time
    'bill_to_address', 'ship_to_address',   # high-cardinality address IDs
    'revision_number', 'status',            # post-order fulfilment state
    'due_date',           # post-order
]
```

```
def filter_cols(cols, drops):
    return [c for c in cols if not any(s in c for s in drops)]

sales_kept = filter_cols(candidate_sales_cols, leakage_or_noise_substrings)
cust_kept  = filter_cols(candidate_cust_cols,  leakage_or_noise_substrings)

print('sales_order_header — kept columns:', sales_kept)
print('sales_order_header — dropped     :', sorted(set(candidate_sales_cols) - set(sales_kept)))
print()
print('customer — kept columns:', cust_kept)
print('customer — dropped     :', sorted(set(candidate_cust_cols) - set(cust_kept)))
```

In [ ]:

In [ ]:
```
feature_selection_2_insights = """
Approach 2 :Anti-leakage column filter.

Inside the two retained tables we still need to drop columns that would
compromise the model:

Leakage columns (carry information that wouldn't exist at prediction time):
  - `ship_date`, `due_date`, `ship_method_id`, `status`, `revision_number`:
    these are populated during/after fulfilment. If we train on them the
    model learns from data we wouldn't actually have at the cutoff.

No-signal columns (high-cardinality IDs and free text):
  - `rowguid`, `modified_date`: system metadata.
  - `sales_order_number`, `purchase_order_number`, `account_number`,
    `credit_card_*`, `currency_rate_id`: high-cardinality transactional IDs.
  - `comment`: free text; would need NLP to use.
  - `bill_to_address_id`, `ship_to_address_id`: each customer can have many
    of these; we use territory_id instead as a coarser geography signal.

Whatever remains after this filter is allowed to participate in the
engineered feature set (Section D).
Note: we are NOT engineering features, yet we are just pruning the universe of raw inputs.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_2_insights', value=feature_selection_2_insights)
```

A.3 Approach 3 — Identifier / near-constant column filter

In [ ]:
```
def column_summary(df, table_name):
    """Helper function to generate summary stats for columns."""
    summary_list = []
    for col in df.columns:
        summary_list.append({
            'table': table_name,
            'column': col,
            'n_rows': len(df),
            'n_unique': df[col].nunique(),
            'pct_top': df[col].value_counts(normalize=True).iloc[0] if not df[col].empty else 0,
            'pct_missing': df[col].isna().mean()
        })
    return pd.DataFrame(summary_list)

# Identifier columns and near-constant columns offer no learnable signal at
# the customer level. Identifiers (e.g. sales_order_id) are PER-ROW unique;
# near-constant columns (e.g. online_order_flag = 0 for ≥99% of rows) are
# essentially fixed values. Both should be excluded as raw inputs, although
# they may still be USED to engineer aggregate features later (e.g. ratio of
# online orders per customer).
diag = pd.concat([
    column_summary(sales[sales_kept], 'sales_order_header'),
    column_summary(cust[cust_kept],   'customer'),
], ignore_index=True)

diag['flag_identifier']    = diag['n_unique'] == diag['n_rows']
diag['flag_near_constant'] = diag['pct_top'] >= 0.99
diag['flag_high_missing']  = diag['pct_missing'] >= 0.50
diag = diag.drop(columns=['n_rows'])

# Assign explicit table order so sales_order_header always precedes customer
diag['_table_order'] = diag['table'].map({'sales_order_header': 0, 'customer': 1})
```

```
display(
    diag.sort_values(
        ['_table_order', 'flag_identifier', 'flag_near_constant', 'flag_high_missing'],
        ascending=[True, False, False, False]
    )
    .drop(columns=['_table_order'])
    .reset_index(drop=True)
)
```

In [ ]:

In [ ]:
```
feature_selection_n_insights = """
Approach 3 — Identifier / near-constant / high-missingness filter.

The diagnostic above tags three classes of low-value column:
  - Identifier columns (`sales_order_id` and similar): unique per row, so
    they cannot generalise. They are kept ONLY to support aggregation in
    Section D (e.g. counting orders per customer) and are dropped from
    the model feature set in Section E.
  - Near-constant columns (top value ≥ 99%): no learnable variance.
  - High-missingness columns (≥ 50% NaN): too much imputation is required
    to recover usable signal.

Output of this step: an explicit allow-list of columns that are RAW INPUTS
to the customer-level aggregation in Section D. The model itself will
never see any of these raw rows, only the per-customer aggregates derived
from them.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_n_insights', value=feature_selection_n_insights)
```

A.z Final Selection of Features

In [ ]:
```
# Raw input columns that survive all three filters and are used to engineer
# the customer-level feature set in Section D. These names are flexible —
# the engineering code uses .get(col) patterns, so columns that don't exist
# in the actual data are simply skipped.
raw_sales_inputs = [
    'sales_order_id',      # used for counting only
    'order_date',          # used for recency / frequency / tenure
    'customer_id',         # join key
    'territory_id',        # geography signal
    'sub_total',           # monetary aggregation
    'total_due',           # monetary aggregation (alternative)
    'tax_amt', 'freight',  # supporting amounts
    'online_order_flag',   # online vs offline ratio per customer
]
raw_customer_inputs = [
    'customer_id', 'person_id', 'store_id', 'territory_id',
]
# Customer-level engineered features that WILL go to the model.
# (These don't exist yet — they're created in Section D.)
features_list = [
    # RFM
    'recency_days', 'frequency_orders', 'monetary_total', 'monetary_avg',
    # Behavioural
    'online_order_ratio', 'distinct_territories',
    # Recent activity
    'orders_last_30d', 'orders_last_90d', 'spend_last_90d',
    # Tenure / cadence
    'tenure_days', 'avg_days_between_orders',
    # Customer type
    'is_individual', 'is_business',
    # Geography (one-hot in Section E)
    'territory_id',
]
print(f'Raw inputs from sales_order_header: {len(raw_sales_inputs)} columns')
print(f'Raw inputs from customer          : {len(raw_customer_inputs)} columns')
print(f'Final engineered features         : {len(features_list)} columns')
```

In [ ]:

In [ ]:
```
feature_selection_explanations = """
Final feature set

Raw INPUTS (used inside the prep pipeline, never seen by the model directly):
  - From sales_order_header: order_date, customer_id, territory_id,
    sub_total, total_due, online_order_flag, sales_order_id (for counting).
  - From customer: customer_id, person_id (→ is_individual flag),
    store_id (→ is_business flag), territory_id.

MODEL features (per customer, engineered in Section D):
  - RFM trio: recency_days, frequency_orders, monetary_total / monetary_avg.
  - Recent activity: orders_last_30d, orders_last_90d, spend_last_90d
    captures momentum near the cutoff date.
  - Behavioural mix: online_order_ratio, distinct_territories.
  - Tenure / cadence: tenure_days, avg_days_between_orders.
  - Customer type: is_individual, is_business (mutually informative; both
    kept so the model can learn the 'neither' case if it exists).
  - Geography: territory_id (one-hot encoded in Section E).

All features are computed from data STRICTLY ON OR BEFORE the global
cutoff date defined in Section C, which is the only thing standing between
this design and target leakage.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_explanations', value=feature_selection_explanations)
```

B. Data Cleaning

B.1 Fixing date types and invalid dates

In [ ]:
```
# Convert order_date to a real datetime (it loads as object from CSV) and
# drop any rows whose date cannot be parsed or whose customer_id is missing.
before = len(sales)
sales['order_date'] = pd.to_datetime(sales['order_date'], errors='coerce')

bad_dates    = sales['order_date'].isna().sum()
missing_cust = sales['customer_id'].isna().sum()

sales = sales.dropna(subset=['order_date', 'customer_id']).reset_index(drop=True)
after = len(sales)

print(f'Rows before               : {before:,}')
print(f'Rows with unparseable date: {bad_dates:,}')
print(f'Rows with missing cust_id : {missing_cust:,}')
print(f'Rows after                : {after:,}')
print(f'Date range                : {sales["order_date"].min()} to {sales["order_date"].max()}')
```

In [ ]:
```
```

In [ ]:
```
data_cleaning_1_explanations = """
Issue: `order_date` arrives from CSV as an object/string column, and the
use case is fundamentally temporal. We need to compare each order date to
the global cutoff. Without a proper datetime conversion, every downstream
operation (cutoff filter, recency, tenure, inter-order interval) would
either fail or produce garbage.

Action: parse with `errors='coerce'`, then drop rows where order_date
could not be parsed or where customer_id is missing. The latter cannot
participate in customer-level aggregation.

Impact: makes time-based feature engineering correct, and ensures the
label and features for every customer can be computed deterministically.
Typical AdventureWorks data has near-zero unparseable rows, so the drop
is a safety measure rather than a meaningful loss of data.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_1_explanations', value=data_cleaning_1_explanations)
```

B.2 Fixing duplicate rows

In [ ]:
```
# Check for exact-duplicate rows and duplicate primary keys in both tables.
sales_dup_rows = sales.duplicated().sum()
cust_dup_rows  = cust.duplicated().sum()

sales_dup_pk = sales['sales_order_id'].duplicated().sum() if 'sales_order_id' in sales.columns el
cust_dup_pk  = cust['customer_id'].duplicated().sum()    if 'customer_id'   in cust.columns  else

print(f'sales_order_header  - exact dup rows  : {sales_dup_rows}')
print(f'sales_order_header  - duplicate PK    : {sales_dup_pk}')
print(f'customer            - exact dup rows  : {cust_dup_rows}')
print(f'customer            - duplicate PK    : {cust_dup_pk}')

# Drop duplicates if any. PKs win — keep the first occurrence.
if 'sales_order_id' in sales.columns:
    sales = sales.drop_duplicates(subset=['sales_order_id'], keep='first').reset_index(drop=True)
if 'customer_id' in cust.columns:
    cust  = cust.drop_duplicates(subset=['customer_id'],   keep='first').reset_index(drop=True)

print(f'\nAfter dedup sales rows: {len(sales):,}, customer rows: {len(cust):,}')
```

In [ ]:
```
```

In [ ]:
```
data_cleaning_2_explanations = """
Issue: duplicate primary keys would inflate per-customer aggregates (a
customer with 5 unique orders could appear to have 6 or 7) and bias every
downstream feature. Exact-duplicate rows are equally dangerous because
they bypass any join-side deduplication.

Action: check duplicate counts on both exact rows and on the primary keys
(`sales_order_id`, `customer_id`); drop with `keep='first'` if any are
found.

Impact: guarantees one-row-per-order in `sales` and one-row-per-customer
in `cust`. All later aggregations (count, sum, mean) are therefore correct
by construction.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_2_explanations', value=data_cleaning_2_explanations)
```

B.3 Fixing implausible / extreme order amounts

In [ ]:
```
# Inspect the distribution of order amounts and identify extreme values that
# would dominate monetary aggregates. We use sub_total as the primary money
# field (excludes tax and freight, so closer to true revenue) and fall back
# to total_due if sub_total is unavailable.
money_col = 'sub_total' if 'sub_total' in sales.columns else 'total_due'
print(f'Using money column: {money_col}')
print(sales[money_col].describe(percentiles=[0.01, 0.25, 0.5, 0.75, 0.95, 0.99]).round(2))

# Flag (do not drop) extreme orders: negatives and the top 0.5 %.
neg_orders = (sales[money_col] < 0).sum()
p995       = sales[money_col].quantile(0.995)
extreme    = (sales[money_col] > p995).sum()
print(f'\nNegative-amount orders         : {neg_orders}')
print(f'Orders above 99.5th percentile : {extreme}  (threshold ≈ {p995:,.2f})')

# Drop negatives (data errors); cap extreme values at the 99.5th percentile.
sales = sales[sales[money_col] >= 0].copy()
sales[money_col] = sales[money_col].clip(upper=p995)

print(f'After cleaning — sales rows    : {len(sales):,}')
print(f'Max {money_col} after cap        : {sales[money_col].max():,.2f}')
```

In [ ]:
```
```

In [ ]:
```
data_cleaning_3_explanations = """
Issue: a small number of orders sit far in the right tail of the amount
distribution. When aggregated to the customer level (sum, mean, max
spend), these extreme values dominate the feature and compress the
discrimination between 'normal' customers. Negative amounts are clear
```

```
data errors (refund rows that should not be summed as revenue).

Action:
  - Drop rows where the money column is negative.
  - Cap the money column at the 99.5th percentile (winsorisation). Capping
    preserves the ordering of customers (a big spender is still a big
    spender) while limiting the influence of any single anomalous order.

Impact: monetary features become more robust and tree-based models become
less sensitive to a handful of outlier customers. Capping affects ≤ 0.5%
of orders so the information loss is small. We deliberately keep the
rows (rather than dropping them) so the customer's frequency and recency
are unaffected.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_3_explanations', value=data_cleaning_3_explanations)
```

B.4 Fixing missing values in retained columns

In [ ]:
```
# Customer-side missing values: `person_id` is NULL for store customers and
# `store_id` is NULL for individual customers — these NULLs are MEANINGFUL
# rather than missing, so we convert them into explicit flags in Section D.
# Anything else missing in the retained columns we impute conservatively.
retained_cust  = [c for c in raw_customer_inputs if c in cust.columns]
retained_sales = [c for c in raw_sales_inputs    if c in sales.columns]

miss_cust  = cust[retained_cust].isna().mean().rename('pct_missing').to_frame()
miss_sales = sales[retained_sales].isna().mean().rename('pct_missing').to_frame()

print('Missingness: customer (retained columns):')
display(miss_cust.style.format({'pct_missing': '{:.2%}'}))
print('Missingness: sales_order_header (retained columns):')
display(miss_sales.style.format({'pct_missing': '{:.2%}'}))

# territory_id missing on the sales side is a small data-quality issue — fill
# from the customer table where possible, otherwise mark as -1 (sentinel).
if 'territory_id' in sales.columns and sales['territory_id'].isna().any():
    sales = sales.merge(cust[['customer_id', 'territory_id']]
                        .rename(columns={'territory_id': 'cust_territory_id'}),
                        on='customer_id', how='left')
    sales['territory_id'] = sales['territory_id'].fillna(sales['cust_territory_id'])
    sales = sales.drop(columns=['cust_territory_id'])
    sales['territory_id'] = sales['territory_id'].fillna(-1).astype(int)
    print('Filled missing sales.territory_id from customer table.')
```

In [ ]:
```

```

In [ ]:
```
data_cleaning_n_explanations = """
Issue: distinguishing genuinely-missing values from structurally-NULL
values is critical for feature quality.
  - `customer.person_id` is NULL by design for store customers, and
    `customer.store_id` is NULL by design for individual customers. These
    NULLs carry information ('this is a store / individual') and must NOT
    be imputed away. They become the `is_individual` / `is_business`
    flags in Section D.
  - `sales.territory_id` missing is a data-quality issue; we fill from the
    customer dimension where possible and use sentinel -1 otherwise. -1 is
    preferred over 'unknown' so the downstream one-hot encoder produces a
    proper indicator column.

Impact: by treating different kinds of NULL differently, we preserve
genuine signal (customer type) while neutralising data noise (missing
geography). Section E imputes any remaining numeric NaNs at split time
using the TRAINING median only, to keep the val/test sets blind.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_n_explanations', value=data_cleaning_n_explanations)
```

C. Split Datasets

In [ ]:
```python
# Section C — Label construction & stratified split
# Design:
#   - Pick a global cutoff = max(order_date) - 90 days.
#   - Features will be built from orders ON OR BEFORE the cutoff (Section D).
#   - Label = 1 if the customer has ≥1 order in (cutoff, max_date], else 0.
#   - Only customers with ≥1 order BEFORE the cutoff are eligible (otherwise
#     we have no features for them).
#   - Split is stratified-random on the label, at the customer level.
#     Because each customer appears exactly once and the cutoff is global,
#     this is leak-free.

from sklearn.model_selection import train_test_split

PREDICTION_WINDOW_DAYS = 90
RANDOM_STATE = 42
TARGET = 'will_reorder'

max_date    = sales['order_date'].max()
cutoff_date = max_date - pd.Timedelta(days=PREDICTION_WINDOW_DAYS)
print(f'Data range          : {sales["order_date"].min().date()} → {max_date.date()}')
print(f'Cutoff (features ≤) : {cutoff_date.date()}')
print(f'Window  (label from): {(cutoff_date + pd.Timedelta(days=1)).date()} → {max_date.date()}')

pre  = sales[sales['order_date'] <= cutoff_date].copy()
post = sales[sales['order_date'] >  cutoff_date].copy()

eligible = pre['customer_id'].unique()
reorderers = set(post['customer_id'].unique())

labels = pd.DataFrame({'customer_id': eligible})
labels[TARGET] = labels['customer_id'].isin(reorderers).astype(int)
print(f'\nEligible customers     : {len(labels):,}')
print(f'Positive class (reorder): {labels[TARGET].sum():,} '
      f'({labels[TARGET].mean():.2%})')

# Attach the customer-dimension columns we kept in Section A.
customer_panel = labels.merge(cust[[c for c in raw_customer_inputs if c in cust.columns]],
                              on='customer_id', how='left')

# 70 / 15 / 15 stratified split at the customer level.
train_df, temp_df = train_test_split(
    customer_panel, test_size=0.30, stratify=customer_panel[TARGET],
    random_state=RANDOM_STATE,
)
val_df, test_df = train_test_split(
    temp_df, test_size=0.50, stratify=temp_df[TARGET],
    random_state=RANDOM_STATE,
)

training_df   = train_df.reset_index(drop=True)
validation_df = val_df.reset_index(drop=True)
testing_df    = test_df.reset_index(drop=True)

print(f'\nSplit sizes  | positive-class rate')
for name, d in [('train', training_df), ('val', validation_df), ('test', testing_df)]:
    print(f'  {name:5s} : {len(d):>6,}  |  {d[TARGET].mean():.2%}')
```

In [ ]:
```python
```

In [ ]:
```python
data_splitting_explanations = """
Strategy: stratified random split at the CUSTOMER level, with a single
GLOBAL temporal cutoff that separates features (≤ cutoff) from label
(> cutoff).

Why temporal cutoff + customer-level split (not random row-level split):
  - A random row-level split would put some of a customer's orders into
    the training set and others into validation/test, the model would
    'see' the same customer in multiple splits, which is leakage.
  - A purely random customer split with no temporal cutoff would build the
    label from the same time period as the features, also leaking.
  - The combination, single cutoff, customer-level split, guarantees
    that every customer's features are strictly older than their label,
    and that no customer's orders appear in two splits.

Why stratify on the label:
```

```
  - Repeat-purchase is typically imbalanced. Stratifying preserves the
    positive-class rate across train / val / test so metric estimates from
    validation transfer cleanly to test.

Why 70 / 15 / 15:
  - 70% for training is a standard default that leaves enough data for
    val and test to give stable metric estimates given the customer count
    in this dataset. Could be tuned later if dataset size dictates.

Alternative considered: time-block split (different cutoffs per split).
  - More robust against temporal drift but reduces effective training
    size and complicates the comparison of baselines and candidate
    models. Documented here as a future-work option.
"""
```

In [ ]:
```
# Do not modify this code
print_tile(size="h3", key='data_splitting_explanations', value=data_splitting_explanations)
```

D. Feature Engineering

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets

try:
  training_df_eng = training_df.copy()
  validation_df_eng = validation_df.copy()
  testing_df_eng = testing_df.copy()
except Exception as e:
  print(e)
```

D.1 New Features — RFM (Recency, Frequency, Monetary)

In [ ]:
```
# Build customer-level RFM features from `pre` (orders on or before cutoff).
# Each split's customers are looked up independently, so each engineered
# feature uses only that customer's own historical orders — no cross-split
# contamination is possible.
money_col = 'sub_total' if 'sub_total' in pre.columns else 'total_due'

rfm_all = pre.groupby('customer_id').agg(
    last_order_date  = ('order_date', 'max'),
    first_order_date = ('order_date', 'min'),
    frequency_orders = ('order_date', 'count'),
    monetary_total   = (money_col,    'sum'),
    monetary_avg     = (money_col,    'mean'),
).reset_index()

rfm_all['recency_days'] = (cutoff_date - rfm_all['last_order_date']).dt.days
rfm_all['tenure_days']  = (cutoff_date - rfm_all['first_order_date']).dt.days
rfm_all = rfm_all.drop(columns=['last_order_date', 'first_order_date'])

def add_rfm(df):
    return df.merge(rfm_all, on='customer_id', how='left')

training_df_eng   = add_rfm(training_df_eng)
validation_df_eng = add_rfm(validation_df_eng)
testing_df_eng    = add_rfm(testing_df_eng)

print('RFM features attached. Training preview:')
display(training_df_eng[['customer_id', TARGET, 'recency_days',
                         'frequency_orders', 'monetary_total',
                         'monetary_avg', 'tenure_days']].head())
```

In [ ]:
```
```

In [ ]:
```
feature_engineering_1_explanations = """
Feature: RFM trio plus tenure.
  - recency_days     = (cutoff - customer's last order date)
  - frequency_orders = count of orders on or before cutoff
  - monetary_total   = total spend on or before cutoff
  - monetary_avg     = mean order value on or before cutoff
  - tenure_days      = (cutoff - customer's first order date)
```

```
Why this matters:
  - RFM is the canonical customer-value framework in retail and is a
    strong predictor of near-future purchase intent. Recency in
    particular is one of the single most powerful churn / repeat signals.
  - Tenure separates 'long-time customer with a long gap' (probably
    churned) from 'recently acquired customer with a long gap' (probably
    hasn't reached their first repurchase yet).Two very different
    propensities that share the same recency value.

Leakage status: all four features are computed only from `pre` (orders
with order_date ≤ cutoff_date), so no future information is used.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_1_explanations', value=feature_engineering_1_expla
```

D.2 New Features — Recent activity (momentum)

In [ ]:
```
# Activity in the windows immediately before the cutoff — captures momentum
# that the all-time RFM aggregates smooth away.
win_30 = cutoff_date - pd.Timedelta(days=30)
win_90 = cutoff_date - pd.Timedelta(days=90)

recent = pre.groupby('customer_id').apply(lambda g: pd.Series({
    'orders_last_30d': (g['order_date'] >= win_30).sum(),
    'orders_last_90d': (g['order_date'] >= win_90).sum(),
    'spend_last_90d' : g.loc[g['order_date'] >= win_90, money_col].sum(),
})).reset_index()

def add_recent(df):
    return df.merge(recent, on='customer_id', how='left')

training_df_eng   = add_recent(training_df_eng)
validation_df_eng = add_recent(validation_df_eng)
testing_df_eng    = add_recent(testing_df_eng)

print('Recent-activity features attached. Training preview:')
display(training_df_eng[['customer_id', TARGET, 'orders_last_30d',
                         'orders_last_90d', 'spend_last_90d']].head())
```

In [ ]:
```
```

In [ ]:
```
feature_engineering_2_explanations = """
Feature: orders_last_30d, orders_last_90d, spend_last_90d.

Why this matters:
  - Aggregate RFM treats a 5-year-old customer and a new customer the
    same way as long as their totals match. Momentum features distinguish
    a customer who is ACTIVE near the cutoff from one whose activity is
    historic.
  - In repeat-purchase prediction, recent orders are a much stronger
    signal than equally-sized old orders. Two windows (30 and 90 days)
    let the model learn a short-term trend.

Choice of windows:
  - 30 days mirrors a typical promotional / billing cycle.
  - 90 days matches the prediction window, symmetry with the label is
    intentional, making the feature directly interpretable as 'did this
    customer do something in a window comparable to the one we're
    predicting?'.

Leakage status: both windows END at the cutoff date, so the features use
only past data.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_2_explanations', value=feature_engineering_2_expla
```

D.3 New Features — Behavioural mix

In [ ]:
```
# Two behavioural features:
#   online_order_ratio   = share of customer's pre-cutoff orders placed online
```

```
#   distinct_territories = number of unique territories the customer ordered from
behaviour_aggs = {}
if 'online_order_flag' in pre.columns:
    behaviour_aggs['online_order_ratio'] = ('online_order_flag', 'mean')
if 'territory_id' in pre.columns:
    behaviour_aggs['distinct_territories'] = ('territory_id', 'nunique')

if behaviour_aggs:
    behaviour = pre.groupby('customer_id').agg(**behaviour_aggs).reset_index()

    def add_behaviour(df):
        return df.merge(behaviour, on='customer_id', how='left')

    training_df_eng   = add_behaviour(training_df_eng)
    validation_df_eng = add_behaviour(validation_df_eng)
    testing_df_eng    = add_behaviour(testing_df_eng)

    cols_to_show = ['customer_id', TARGET] + list(behaviour_aggs.keys())
    print('Behavioural features attached. Training preview:')
    display(training_df_eng[cols_to_show].head())
else:
    print('No behavioural source columns available; section skipped.')
```

In [ ]:

In [ ]:
```
feature_engineering_3_explanations = """
Feature: online_order_ratio, distinct_territories.

Why this matters:
  - online_order_ratio captures channel preference. Online customers
    typically have higher re-purchase rates than offline customers in
    AdventureWorks-style data because the channel itself lowers friction.
  - distinct_territories indicates customer mobility / multi-location
    use. A customer ordering from many territories often represents a
    business account with multiple sites. A different repeat-purchase
    pattern than a single-location individual.

Both are RATIO / COUNT aggregates of pre-cutoff orders, so they leak no
future information.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_3_explanations', value=feature_engineering_3_expla
```

D.4 New Features — Customer type and order cadence

In [ ]:
```
# 1) Customer type flags from the `cust` dimension NULL pattern (Section B.4).
training_df_eng['is_individual']  = training_df_eng.get('person_id', pd.Series(np.nan, index=trai
training_df_eng['is_business']    = training_df_eng.get('store_id',  pd.Series(np.nan, index=trai
validation_df_eng['is_individual'] = validation_df_eng.get('person_id', pd.Series(np.nan, index=v
validation_df_eng['is_business']   = validation_df_eng.get('store_id',  pd.Series(np.nan, index=v
testing_df_eng['is_individual']    = testing_df_eng.get('person_id', pd.Series(np.nan, index=test
testing_df_eng['is_business']      = testing_df_eng.get('store_id',  pd.Series(np.nan, index=test

# 2) Average inter-order gap (cadence). Only customers with ≥2 orders have
#    a meaningful value; we fill the rest with NaN, to be imputed in E.
def cadence(g):
    if len(g) < 2:
        return np.nan
    diffs = g.sort_values('order_date')['order_date'].diff().dropna().dt.days
    return diffs.mean()

cadence_df = (pre.groupby('customer_id').apply(cadence)
                 .rename('avg_days_between_orders').reset_index())

def add_cadence(df):
    return df.merge(cadence_df, on='customer_id', how='left')

training_df_eng   = add_cadence(training_df_eng)
validation_df_eng = add_cadence(validation_df_eng)
testing_df_eng    = add_cadence(testing_df_eng)

print('Final engineered training preview:')
display(training_df_eng.head())
print(f'\nFinal training shape: {training_df_eng.shape}')
```

In [ ]:

In [ ]:
```
feature_engineering_n_explanations = """
Features: is_individual, is_business, avg_days_between_orders.

Customer type flags:
  - In AdventureWorks, `customer.person_id` is populated only for
    individuals and `customer.store_id` is populated only for businesses.
    Converting the NULL pattern into two binary flags preserves that
    structural information for the model.

Average inter-order gap (cadence):
  - Customers fall into very different cadences: weekly, monthly,
    quarterly, yearly. A customer whose typical gap is 30 days and whose
    recency is 45 days is overdue; a customer whose typical gap is 120
    days and whose recency is 45 days is right on schedule. The cadence
    feature is therefore complementary to recency.
  - Single-order customers have no defined cadence; left as NaN and
    imputed in Section E with the training-set median (a conservative
    'typical' cadence).

All three features are computed from pre-cutoff data only,no leakage.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_n_explanations', value=feature_engineering_n_expla
```

E. Data Preparation for Modeling

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets

try:
  X_train = training_df_eng.copy()
  X_val = validation_df_eng.copy()
  X_test = testing_df_eng.copy()
except Exception as e:
  print(e)
```

E.1 Data Transformation — Drop non-model columns

In [ ]:
```
# Drop columns that exist only to support feature engineering / merges and
# are NOT meant to feed the model. customer_id stays out of features but is
# preserved separately as an index reference if needed.
non_model_cols = ['customer_id', 'person_id', 'store_id']

for df_name in ['X_train', 'X_val', 'X_test']:
    df = globals()[df_name]
    drop_here = [c for c in non_model_cols if c in df.columns]
    globals()[df_name] = df.drop(columns=drop_here)

print(f'X_train columns ({X_train.shape[1]}):', list(X_train.columns))
print(f'\nX_train shape: {X_train.shape}')
print(f'X_val   shape: {X_val.shape}')
print(f'X_test  shape: {X_test.shape}')
```

In [ ]:

In [ ]:
```
data_transformation_1_explanations = """
Action: drop the ID-style columns (`customer_id`, `person_id`, `store_id`)
from the feature matrices.

Why:
  - `customer_id` is a unique identifier; including it would let a tree
    model memorise individual customers and create artificial accuracy
    that doesn't generalise.
  - `person_id` and `store_id` were only kept to derive `is_individual`
    and `is_business` in D.4; their raw values are still high-cardinality
    nuisance variables.
```

```
We keep TARGET (`will_reorder`) in the X frames for now. It's separated
out in section E.4 so y can be saved to disk alongside X.
"""
```

E.2 Data Transformation — Imputation of remaining NaNs

In [ ]:
```python
from sklearn.impute import SimpleImputer

# Column groups
num_cols = [c for c in X_train.columns
            if c != TARGET
            and pd.api.types.is_numeric_dtype(X_train[c])
            and c not in {'territory_id'}]
cat_cols = [c for c in ['territory_id'] if c in X_train.columns]

#  Numeric imputation (median, fit on train only)
num_imp = SimpleImputer(strategy='median').fit(X_train[num_cols])
for df_name in ['X_train', 'X_val', 'X_test']:
    df = globals()[df_name]
    df[num_cols] = num_imp.transform(df[num_cols])
    globals()[df_name] = df

# Categorical encoding (label-encode, fit on train only)
# Build a {raw_value → int_code} map from training data.
# NaN and any unseen value in val/test are mapped to -1 (sentinel).
cat_maps = {}
for col in cat_cols:
    unique_vals = sorted(X_train[col].dropna().unique())          # fit on train
    cat_maps[col] = {v: i for i, v in enumerate(unique_vals)}    # 0-based codes

for df_name in ['X_train', 'X_val', 'X_test']:
    df = globals()[df_name]
    for col in cat_cols:
        df[col] = df[col].map(cat_maps[col]).fillna(-1).astype(int)
        # .map() returns NaN for unseen / missing → fillna(-1) catches both
    globals()[df_name] = df

# Verification
print('Numeric imputation values (training median):')
display(pd.Series(num_imp.statistics_, index=num_cols).round(3))

if cat_cols:
    print('\nCategorical encoding cardinalities (train):')
    for col, m in cat_maps.items():
        print(f'  {col}: {len(m)} known categories → codes 0–{len(m)-1}, sentinel -1')

print('\nRemaining NaNs across splits (should be 0):')
print(f'  X_train: {X_train.isna().sum().sum()}')
print(f'  X_val  : {X_val.isna().sum().sum()}')
print(f'  X_test : {X_test.isna().sum().sum()}')
```

In [ ]:
```python
```

In [ ]:
```python
data_transformation_2_explanations = """
Action: median-impute numeric NaNs and sentinel fill territory_id NaNs.
Imputation statistics are fit on the TRAINING set only.

Why:
  - Median is preferred over mean for monetary and cadence features
    because their distributions are heavily right-skewed even after the
    99.5% cap in B.3.
  - Fitting on training only is non-negotiable for honest evaluation:
    using val/test means to fill train values would leak information
    from the held-out sets into training.
  - territory_id is treated as categorical: -1 becomes its own
    one-hot column in the next step rather than being collapsed into a
    real territory.
"""
```

In [ ]:
```python
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_2_explanations', value=data_transformation_2_expla
```

E.3 Data Transformation — Encoding and scaling

In [ ]:
```python
# One-hot encode territory_id (low cardinality) and standardise numeric
# features. Fit on training only; apply to val/test.
from sklearn.preprocessing import StandardScaler

def one_hot_align(train, val, test, cols):
    train_oh = pd.get_dummies(train, columns=cols, prefix=cols, dtype=int)
    val_oh   = pd.get_dummies(val,   columns=cols, prefix=cols, dtype=int)
    test_oh  = pd.get_dummies(test,  columns=cols, prefix=cols, dtype=int)
    # Align val/test columns to training schema (drop unseen, add missing).
    val_oh  = val_oh.reindex(columns=train_oh.columns, fill_value=0)
    test_oh = test_oh.reindex(columns=train_oh.columns, fill_value=0)
    return train_oh, val_oh, test_oh

if cat_cols:
    X_train, X_val, X_test = one_hot_align(X_train, X_val, X_test, cat_cols)
    print(f'After one-hot, X_train has {X_train.shape[1]} columns.')

# Refresh num_cols (still the same — one-hot only added new columns).
scaler = StandardScaler().fit(X_train[num_cols])
X_train[num_cols] = scaler.transform(X_train[num_cols])
X_val[num_cols]   = scaler.transform(X_val[num_cols])
X_test[num_cols]  = scaler.transform(X_test[num_cols])

print(f'Final feature counts — train: {X_train.shape[1]-1} features + 1 target')
display(X_train.head())
```

In [ ]:
```python
```

In [ ]:
```python
data_transformation_3_explanations = """
Action: one-hot encode territory_id; standardise (z-score) numeric features.
Both encoders/scalers are fit on TRAINING data only and applied to val/test.
Val/test column schemas are re-aligned to training in case a territory
appears in one split but not another.

Why one-hot for territory:
  - Territory is nominal, no natural ordering. Tree-based models tolerate
    integer encoding but linear models (logistic regression) would treat
    territory 5 as 'closer' to territory 6 than to territory 1, which is
    meaningless.

Why z-score scaling:
  - Regularised linear models (used in the classification notebook for
    benchmarking) are scale-sensitive; without scaling, monetary_total
    dominates because it ranges over thousands while ratios range over
    0–1.
  - Tree-based models (RF, GBM) don't need scaling but are unaffected by
    it. Scaling once here means downstream models can be swapped without
    revisiting preprocessing.
"""
```

In [ ]:
```python
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_3_explanations', value=data_transformation_3_expla
```

E.4 Save target arrays (y_train / y_val / y_test)

In [ ]:
```python
# Separate the target from each feature matrix and write the y_*.csv files
# that the Baseline and Classification notebooks expect to read back.
# We do this HERE (rather than in section F) because section F is the
# protected save block for X_*.csv only.
y_train = X_train.pop(TARGET)
y_val   = X_val.pop(TARGET)
y_test  = X_test.pop(TARGET)

print('Final shapes after splitting features from target:')
print(f'  X_train: {X_train.shape}   y_train: {y_train.shape}')
print(f'  X_val  : {X_val.shape}     y_val  : {y_val.shape}')
print(f'  X_test : {X_test.shape}    y_test : {y_test.shape}')

y_train.to_frame(TARGET).to_csv(at.folder_path / 'y_train.csv', index=False)
y_val.to_frame(TARGET).to_csv(at.folder_path / 'y_val.csv', index=False)
y_test.to_frame(TARGET).to_csv(at.folder_path / 'y_test.csv', index=False)

print('\nSaved y_train.csv, y_val.csv, y_test.csv to the assignment folder.')
print('Section F will now save X_train.csv, X_val.csv, X_test.csv (target-free).')
```

In [ ]:

In [ ]:
```
data_transformation_n_explanations = """
Action: pop the target column out of X_train / X_val / X_test, save it to
y_train.csv / y_val.csv / y_test.csv in the assignment folder. The X
frames are now ready to be saved by the protected code in Section F.

Why here and not in Section F:
  - Section F is the assignment-provided save block and we are instructed
    not to modify it. By popping the target and writing the y files in
    Section E, the protected X save in Section F runs on target-free
    feature matrices, exactly as expected by the Baseline and
    Classification notebooks.

Output contract:
  - X_*.csv : numeric, target-free feature matrices ready for sklearn.
  - y_*.csv : single column named 'will_reorder' with values in {0, 1}.
"""
```

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_n_explanations', value=data_transformation_n_expla
```

F. Save Datasets

> Do not change this code

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL

try:
  X_train.to_csv(at.folder_path / 'X_train.csv', index=False)

  X_val.to_csv(at.folder_path / 'X_val.csv', index=False)

  X_test.to_csv(at.folder_path / 'X_test.csv', index=False)
except Exception as e:
  print(e)
```

G. Ethics — pipeline-level safeguards

In [ ]:
```
ethics_pipeline_explanations = """
Ethics: privacy-protective design choices in the prep pipeline

Choices already made in this pipeline that reduce privacy risk:

  1. customer_id, person_id and store_id are DROPPED from the feature
     matrix before modelling (E.1). The model cannot memorise individuals
     because the model literally never sees them.
  2. No free-text, address, contact or payment fields enter the feature
     matrix. The columns dropped in A.2 (rowguid, account_number, credit
     card, comment, ship/bill address) are excluded by design.
  3. Aggregations are at the CUSTOMER level (RFM, recent activity,
     behavioural mix). The model output is one row per customer; no order-
     level details leak into the scoring table.
  4. Imputers, encoders and scalers are FIT ON TRAINING ONLY. This is a
     correctness measure (no leakage) but it also limits the propagation
     of held-out customer statistics.
  5. The 99.5th-percentile winsorisation reduces the influence of any
     single anomalous customer on monetary aggregates, which mitigates
     membership-inference risk in the score distribution.

Choices for the operator (NOT made by this pipeline):
  - Retention period for X_train / X_val / X_test on disk.
  - Access control on the saved CSVs.
  - Opt-out enforcement at the targeting layer (not just the email-send
    layer).
  - Indigenous-data-governance arrangements where applicable
    (AIATSIS Code of Ethics 2020; CARE principles).

These operator choices are out of scope for the notebook but must be
documented in the deployment plan before the pipeline runs in production.
"""
```

```
print_tile(size="h3", key='ethics_pipeline_explanations', value=ethics_pipeline_explanations)
```